Le jeu *Hex* – TP2 – Jouer « contre l'ordinateur »

Ce TP fait suite au précédent : reprenez le code que vous avez réalisé au TP1.
Il s'agit maintenant de permettre de jouer « contre l'ordinateur ». Dans le paquetage `hex.ia`, il y a trois classes qui implémentent l'interface `StrategieIA`, et qui proposent chacune une implémentation différente de la méthode `choisirCase`. Cette méthode permet de sélectionner automatiquement une case où placer un jeton, en fonction de l'état du plateau et du joueur actif. Cela nous permettra de jouer « contre l'ordinateur ».

1. Vérification de votre avancement

1. Lancez la classe Lanceur et faîtes une partie contre vous-même sur un plateau de Hex de taille 6.
2. Corrigez les éventuels problèmes avant de passer à la question suivante.

2. Jouer « contre l'ordinateur »

Dans un premier temps, nous allons modifier notre code pour que le joueur 2 soit le joueur « ordinateur ».

1. Consultez la documentation du paquetage `hex.ia`.
2. Dans la classe `Gestionnaire`, modifiez la boucle de jeu de telle sorte que :
   — Le choix du joueur 1 soit demandé comme précédemment, avec `ihm.demanderCase` ;
   — Le choix du joueur 2 soit calculé avec la méthode `choisirCase` de la classe `StrategiePCC`.
3. Testez votre code en effectuant une partie sur un plateau de taille 6x6.

3. Choisir la difficulté

Les trois classes du paquetage `hex.ia` correspondent à trois niveaux de difficulté (`StrategiePCC` = très facile ; `StrategiePCCCritique` = facile ; `StrategiePCCInter` = moyen). Nous allons permettre à l'utilisateur de choisir la difficulté du joueur « ordinateur ».

1. Rappelez ce qu'est le patron de conception *Stratégie*
2. Dans la classe `Gestionnaire`, créez un attribut `strat2` de type `StrategieIA` et ses getter/setter.
3. Modifiez la boucle de jeu pour que le choix du joueur 2 soit effectué avec `strat2.choisirCase` si `strat2` est différent de `null`, et avec `ihm.demanderCase` sinon.
4. Dans la méthode `main`, utilisez le setter `setStrat2` avant la création de la partie afin d'utiliser la stratégie `StrategiePCCInter`
5. Testez votre code en effectuant une partie sur un plateau de taille 6x6.

4. Améliorer l'IHM

La classe `Menu` du paquetage `hex.affich` permet d'afficher un menu et de récupérer le choix de l'utilisateur. Nous allons modifier notre *Facade* `IHMConsole` pour permettre le choix de la difficulté via un menu.

1. Ajoutez à l'interface `IHM` la spécification suivante : `public StrategieIA demanderStrategieJoueur2();`
2. Implémentez cette méthode dans la classe `IHMConsole`, en vous reposant sur le service fourni par `Menu.promptChoix`. Les items du menu sont "Joueur humain", "IA (très facile)", "IA (facile)" et "IA (moyen)". Votre méthode renverra `null` pour un joueur humain ou une instance de `StrategiePCC`, `StrategiePCCCritique` ou `StrategiePCCInter` sinon.
3. Dans la classe `Gestionnaire`, modifiez la méthode `nouvellePartie` pour qu'elle demande et mémorise la stratégie du joueur 2.
4. Dans le `main`, supprimez l'appel à `setStrat2` et testez votre code sur un plateau de taille 6x6.

Remarque : Votre code permet maintenant au joueur 1 de choisir s'il joue contre un humain ou contre l'ordinateur (et la difficulté). Si vous souhaitez faire s'affronter deux IA, il faut refaire les questions 3 et 4 pour le joueur 1.

5. Ajouter de nouvelles stratégies IA

Le patron de conception *Stratégie* permet par ailleurs de respecter le principe Ouvert/Fermé (et les principes de responsabilité unique et d'inversion des dépendances) : on peut facilement ajouter de nouvelles stratégies.

1. Créez une classe `StrategieHasard` implémentant l'interface `StrategieIA` et codez-y la stratégie suivante : une case libre est sélectionnée au hasard.
2. Ajoutez le niveau de difficulté "extrêmement facile" correspondant à cette stratégie.

Considérations sur les algorithmes utilisés : Les trois implémentations de `StrategieIA` fournies reposent sur le même algorithme cherchant les cases situées sur les plus courts chemins reliant les bords opposés (calculés en nombre de jetons manquants). Cependant, l'algorithme utilisé n'est pas un algorithme de graphe, mais une propagation des distances sur les cases du plateau. Par exemple, si le joueur actif est le joueur 1 (jeton `@`), on calcule d'abord les distances minimales à partir du bord gauche par itérations successives (c'est-à-dire le nombre de jetons minimal à poser pour relier une case donnée au bord gauche) :

```
    - - X - - -    1 - X - - -    1 2 X - - -    1 2 X - - -    1 2 X - - -
    - - X - - -    1 - X - - -    1 2 X - - -    1 2 X - - -    1 2 X - - -
    - - @ X X -    1 - @ X X -    1 2 @ X X -    1 2 @ X X -    1 2 @ X X -
  @ - - @ - -    @ 1 - @ - -    @ 1 2 @ - -    @ 1 2 @ 3 -    @ 1 2 @ 3 4
 - - - - @ -    1 1 - - @ -    1 1 2 - @ -    1 1 2 3 @ 3    1 1 2 3 @ 3
- - - - - - -    1 - - - - -    1 2 2 - - -    1 2 2 3 3 3    1 2 2 3 3 3
E. Initial       Etape 1        Etape 2        Etape 3        Etape 4        etc.
```

On calcule de la même manière les distances au bord droit. Puis, on additionne ces distances : les cases ayant les valeurs minimales (ici, 4) appartiennent toutes à au moins un des plus courts chemins entre les bords gauche et droit.

```
    1 2 X 7 7 7      3 3 X 3 2 1      4 5 X n 9 8      + - X - - -
    1 2 X 7 6 6      3 2 X 3 2 1      4 4 X n 8 7      + + X - - -
    1 2 @ X X 5      3 2 @ X X 1      4 4 @ X X 6      + + @ X X -
  @ 1 2 @ 3 4   +  @ 3 2 @ 2 1   =  @ 4 4 @ 5 5   ->  @ + + @ - -
 1 1 2 3 @ 3      5 4 3 2 @ 1      6 5 5 5 @ 4      - - - - @ +
1 2 2 3 3 3      6 5 4 3 2 1      7 7 7 7 5 4      - - - - - +
Distance ->      Distance <-        Somme        Cases identifiées
```

Puisque chaque case est considérée une seule fois lors de la propagation, cet algorithme est "efficace" (sa complexité est linéaire en le nombre de cases) et donc adapté même pour de grands plateaux de jeu. Les trois stratégies diffèrent par la sélection finale de la case :

- L'algorithme de `StrategiePCC` sélectionne l'une des cases identifiées au hasard ;
- L'algorithme de `StrategiePCCCritique` sélectionne l'une des cases "critiques" (dans un goulet d'étranglement) ;
- L'algorithme de `StrategiePCCInter` effectue le calcul pour les deux joueurs et sélectionne une des cases à l'intersection des plus courts chemins des deux joueurs (donc, avantageuse pour le joueur actif mais aussi gênante pour l'adversaire).

Ces stratégies ne sont pas très bonnes : elles correspondent à des niveaux de difficulté "très facile", "facile" et "moyen", respectivement.